Some personal tips and C Programming Fundamentals

Comprehensive Notes on Kernighan & Ritchie

1 Some general tips that leaked into this doc

- **Modern SWE & LLMs:** LLMs are invaluable tools, capable of both practical execution and high level planning. Still, the latter seems more promising as an area where humans can remain essential...not only is it hard to decide what to build, high level architectural decisions require significant human expertise and context on the requirements of a project. Essentially, we are still necessary for the core task of engineering: decision-making regarding tradeoffs. Once we've got a design doc ready, AI can do much of the implementation work...although it's not exactly perfect for that either.

- **Techniques to look like a good programmer:** Print/fstring/format strings, try/catch & exception handling (target explicit anticipated errors vs generalized catch), enums, switch statements (good practice is to add a default case), ternary operators (if x then y else z from Python or (condition) ? true action : false action), general idioms e.g. Pythonic code, languages are made by humans with intended optimizations/practices to speak to the machine

- **Encapsulation:** Easy to discuss but hard to put into practice...a key design pattern where we shield code from other code's specifics to make future changes easier. Never make assumptions about underlying implementations. Interact purely with interfaces utilizing known inputs and outputs ("I put X in, I get Y out"). Requires foresight but can be very powerful and encourages generic programming.

- **Main MAJOR ERRORS a SWE should avoid:** Implementations can mainly fail on security, cost, reliability, efficiency, and usability.

- **Exception Handling:** Always anticipate exceptions (`try/catch`), even in asynchronous flows.

- **Architecture Review (CS61C concepts):**
  - 1 byte = 8 bits = 2 hex digits (1 hex digit directly converts to 4 bits).
  - An instruction set maps directly to hardware designed to execute its particular operations. As such assembly and actual CPU hardware are directly connected and the source of headaches regarding some computers being physically unable to execute compiled programs intended for machines with alternative architectures.
  - **IEEE 754 Floating Point:** Comprised of sign, exponent, and fraction bits. Assumes a normalized 1.XXXX mantissa unless encountering a special case (e.g., exponent = 0).

- **Concurrency from 61C:** The main danger is race conditions, unfortunately not all work is easy to parallelize/split into independent parts. This class involves surrendering this challenge to the OpenMP library for simple tasks like for loops but deeper techniques involve locks (risk

of deadlock) and atomic operations (instruction fully completes in one go so never worry abt stale data/order conflict).

- **Microcontrollers/Embedded** Microcontrollers and even RTOS are just stripped down computers/operating systems for specific use cases. They receive very basic commands compiled down to hardware-specific assembly. Apparently there's something called a hypervisor sitting between the OS and hardware to assign hardware to multiple OS's...so many middlemen. Note there isn't a hypervisor IN these embedded applications, more of a "shower thought" here.

2 Chapter 1: Basics

- **Compilation & Preprocessor:** The `#include <stdio.h>` directive imports header files granting access to standard I/O like `printf`. Compilation utilizes tools like `gcc` (e.g., `gcc -o`) or modern build systems like CMake.
- **Program Structure:** The `main()` function dictates execution. Arithmetic operations (`*=`, `/=`) and multiline comments (`/* ... */`) follow standard syntax.
- **Memory Footprints:**
  - `char`: 1 byte (8 bits)
  - `int`: 4 bytes (32 bits)
  - `float`: 32-bit (IEEE-754 precision)
  - `double`: 64-bit precision
- **Memory Model:**
  - Code/text stores string literals and processed instructions.
  - The stack stores local variables, 'expands downwards' and provides functions with space to store intermediate values as they compute desired outputs.
  - The heap provides dynamic memory for malloc operations to utilize providing 'global' memory by allowing pointers to be tied to heap addresses which enables data to persist beyond e.g. a function's local scope. Must be freed later.
  - The data segment stores global and static variables.
- **Truncation Danger:** Integer division strictly truncates fractional results. Evaluating `(5/9) * x` will reliably evaluate to `0` because $5/9 \to 0$.
- **Format Specifiers:** `printf`, `fprintf`, and `scanf` consume arguments off the stack (CS161/x86 behavior, this might be different for modern architectures e.g. Apple Silicon's ARM chips). You *must* supply all arguments matching the specifiers: `%d` (integer), `%x` (hex byte), `%p` (pointer), `%c` (char), `%u` (unsigned int), `%s` (string).
- **Macros:** `#define LOWER 10` creates early constants for the file, saving memory. Often capitalized by convention. Conceptually similar to Scheme macros, building complex repeating logic from simple instructions.
- **Loops & Logic:** Standard `for`, `while`, and `do-while` structures. Maintain a strict distinction between logical operators (`||`, `&&`, `!`) and bitwise operators (`|`, `&`, `~`).

- **Raw Arrays:** A central data structure in C, deal with these very often to store data, no fancy/easy resizing and no automatic out of bounds checks. Also the null terminator at the end for strings, an essential symbol.

3 Chapter 2: Types, Operators, and Expressions

- **Constants & Literals:** `0x41` is the ASCII hex code for 'A'. String literals are inherently constant. Attempting to alter the contents of a string literal directly will result in a segmentation fault. Use `const` frequently to catch logic errors and improve compiler optimization.
- **Type Modifiers:** Applying `short`, `long`, `signed`, or `unsigned` to an `int` allows memory optimization and strict boundary enforcement (e.g., forcing strictly positive values gives an extra magnitude bit). Remember: memory vs accuracy tradeoff and advanced floating point behavior vs simple fixed point math, this is a common decision point for real projects.
- **Type Conversion:** `atoi` converts a string/char to an `int`. ASCII manipulation is just math (e.g., `+ 'a' - 'A'` to shift case).
- **Enums:** Enumerations centralize information by defining a strict list of values. For example, if a game character has 4 distinct classes, defining them in an enum is significantly safer and easier to update than disjointed `#define` macros.
- **Bitwise Operations:** Crucial for low-level logic:
  - `&` (AND): Often used with 0 to clean/mask bits.
  - `|` (OR): Used to acquire or turn on bits.
  - `^` (XOR), `«` (Left Shift), `»` (Right Shift), `~` (One's Complement).
- **Unary Operators:** Computes new value and assigns it to original variable, very valuable for cleaning up code and requires some decision making for preceding (e.g. `++x`) or following (e.g. `x++`) the statement's other operations.
- **Ternary Operator:** `condition ? true_action : false_action`.

4 Chapter 3 & 4: Control Flow, Functions, and Scope

- **Statements:** The semicolon (`;`) terminates a statement. Be careful with deeply nested `if-else` trees.
- **Switch:** `switch(expr)` routes logic through specific `case` constants. Requires a `break` to prevent fall-through. Good practice to add a default case.
- **Scope Rules:** Local variables declared inside a function reside on the stack and are invisible outside that function.
- **Header Files:** Provides information about what functions/info is available to the code to actually complete operations, potentially some implementation details to help the compiler, can have complex dependencies e.g. transitive so sequence of headers requiring others to function properly, etc.

- **Storage Classes:** The `register` keyword suggests to the compiler that a highly utilized variable be stored directly in a CPU register for execution speed.
- **Char Array:** Special array initialization char *name to create a string literal (read-only in code/text segment, error risk); contrasts with char name[] = 'string'; which creates a valid mutable array on the stack.

5 Chapter 5: Pointers and Arrays

Pointers are the core of C's memory management. A pointer stores a memory address (e.g., 32-bit or 64-bit `0x???????`).

- **Basics:** `void *` is a generic placeholder pointer. `&` extracts an address (e.g., `int *p = &x;`). `*` dereferences the pointer to read or update the value (e.g., `*p = 5;`).
- **Pointer Arithmetic:** Array indexing is syntactic sugar for pointer math, i.e. it's a simpler way to represent the same operation which is more amenable to human users, as we're the ones who code (AI tho...). `a[i]` mathematically equates to `*(a + i)`. If `pa` is a pointer to an array element, `*(pa+1)` advances to the next element, which involves an implicit calculation to advance by the size of one of pa's elements.
- **Classic example of using pointers to achieve Pass-by-Reference behavior:** To alter a caller's variables, you must pass pointers; technically you're still passing in values, these values just happen to be memory addresses which enable updates to external variables pointing to the same location. Additionally this enables us to update variables in a broader scope from within a function.

```c
// Standard Swap idiom
void swap(int *px, int *py) {
    int temp = *px;
    *px = *py;
    *py = temp;
}

```

- **Heap Memory (`malloc`/`free`):**
  - `malloc` requests specified space.
  - `calloc` allocates space and safely zero-initializes it (slower but prevents garbage data/missing null terminators).
  - **CRITICAL:** Memory must be `free`'d. **Never** access memory after it is freed. You technically can but it's a classic safety issue since that memory could contain anything including hostile or garbage data.
- **Stack Address Risks:** Never return a pointer assigned to a local variable's stack address. Once the function finishes execution and goes out of scope, the memory space is automatically reclaimed, rendering the pointer invalid.
- **The absolute classic `strcpy` Idiom:**

```c
void strcpy(char *s, char *t) {
    while ((*s++ = *t++) != '\0');
}

```

This highly compact logic dereferences `s` and `t`, copies the value from `t` to `s`, evaluates the `!= '\0'` condition, and *then* increments both pointers post-operation. It's characteristic of C's design: great freedom and potential for clean code for a careful programmer.

- **Pointer Arrays:** Be aware of `char *argv[]` or `char *name[]`, which are incredibly efficient for backing up a large group of variable-length items (like strings) without allocating contiguous multi-dimensional arrays (e.g., `daytab[2][13]`). item **Built-in functions:** Several string manipulation tools like strcmp, strstr, substr, and printf's physical spacing arguments.
- **Function Pointers:** Pointers can reference executable functions for generic operations: `int (*comp)(void*, void*)`. Parentheses are strictly necessary due to operator precedence (`int *pf()` implies a function returning a pointer, whereas `int (*pf)()` is a pointer to a function).
- **Tooling:** `gdb` (`x/16x` to inspect memory blocks) and `Valgrind` for tracking memory leaks. These two are essential for debugging, tracking program state, and directly supervising memory management.

6 Chapter 6: Structures and Unions

- **Structures:** Fundamentally a group of bundled variables which make sense to store together. Perhaps the predecessor of a class (no unique methods, inheritance, or polymorphism yet). Access via dot (`.`) or arrow (`->`) if accessing through a pointer. Pointer member incrementing (`++p->len`) relies on specific evaluation order precedence.
- **Defined comparisons:** `#define min(a, b) ((a) < (b) ? (a) : (b))` allows us to create a new function which can be valuable for struct behavior. However be careful with writing these kinds of macros as they could experience undefined behavior in certain edge cases e.g. interactions with unary operators like a call including x++ as the first argument, causing x to increment twice.
- **Typedef:** Creates aliases for types (e.g., `typedef int Length;`). The declared alias type name goes at the *end* of the statement. A simple way to make it easier to reference a particular type.
- **Self-Referential Structures:** Foundational for binary trees and linked lists, as in the below example. A struct cannot contain itself, but it can contain a pointer to its own type. Remember the NULL root and NULL being identical to 0 as "falsey" values, although NULL is preferred explicitly in this scenario.

```c
struct tnode {
    char *word;
    int count;
    struct tnode *left;
    struct tnode *right;
};

```

Nodes must be dynamically allocated (e.g., a custom `talloc` wrapping `malloc(sizeof(struct tnode))`) and recursively freed. See https://sssd.io/contrib/talloc.html which allows us to allocate memory to store this tree while respecting its recursive structure for assignment and freeing.

- **Unions:** Can hold a variety of object types, but only one is active at a given time. The name doesn't really align exactly with the mathematical construct as it only holds one element but it can be useful for saving on memory and ensuring exclusive fields. Additionally the nature of this "shared memory location" means writing to a new member overwrites any existing data, which can be useful e.g. for tests of endianness.

- **Bit-fields & Flags:** Enums and bitwise operators combine to manage flags.
  - `enum { KEYWORD = 01, EXTERNAL = 02 }` (Powers of 2 for clean bit shifts).
  - `flags |= EXTERNAL | STATIC` guaranteed to turn both bits ON from an initial 0 state.

7 Chapter 7 & 8: Input/Output and UNIX Interfaces

- **File Handling:** Interaction requires a file pointer (`FILE *fp = fopen(name, mode)`). Files must be closed via `fclose()`. Uses functions like `getc(fp)` and `putc()`. Read via `getline/readlines` like Python which reads until a newline character is encountered. Can create macros for automating combinations of repeated instructions. Fgets to read into a buffer (overflows are a classic bug with many tricks and countermeasures, stack canaries and ASLR vs reading off other values to determine the needed addresses...endless arms race).

- **UNIX Integrations:** Direct file I/O redirection using `<` (input) and `>` (output). Familiar commands map cleanly to file system operations (`ls`, `cd`, `grep`, `cat`). The UNIX filesystem also allows for formulas such as ./abcd/*/efgh and other operations, including repeated recursive paths like ../../../../ to step up, go back in, etc, etc. This can be used for attacks e.g. hard to parse where a path will go before the operation occurs and is able to damage your filesystem. To be honest I faded this chapter since it's pretty dense but UNIX operating systems are a cool topic I'll probably look into further via OSTEP.

- **System Calls:** The `system(char *s)` function executes a command at the OS level (high risk, monitor exit statuses). This can also be used for attacks (CS161 dominating this section...good to be conscious of risks! The actual intestines of this stuff are very messy...but also very cool and essential to the machines we use daily!).

- **Standard Libraries:** `stdlib.h`, `math.h` (provides additional operations, RNG, etc), `assert.h` (for testing), `ctype.h`.

8 Appendices

- There's quite a bit more to this part but the below chunk is what I've noted down in particular as a general review. Some components of the appendices are sorta repetitive from earlier parts of the book but it's a valuable reference in case you want to know the full scope of the language.

- The whole language! Programming is ultimately a task of stringing together operations belonging to a closed set to achieve desired behavior, so it's essential to understand the available tools before you start building. With just a few core blocks C can achieve practically anything! (Not super familiar with Turing completeness but yeah...things like actual hardware are probably a bigger obstacle than deep computational logic, but who knows what you'll run into!)

8.1 Reserved Keywords

The following 32 identifiers are reserved as keywords in C and cannot be used otherwise. Understanding what they do is essential:

`auto`	`double`	`int`	`struct`
`break`	`else`	`long`	`switch`
`case`	`enum`	`register`	`typedef`
`char`	`extern`	`return`	`union`
`const`	`float`	`short`	`unsigned`
`continue`	`for`	`signed`	`void`
`default`	`goto`	`sizeof`	`volatile`
`do`	`if`	`static`	`while`

- auto is implied and rarely used.
- double, int, short, float, unsigned, signed are options for number representation and should be chosen based on memory needs and if floating point arithmetic could lead to erroneous or (conventionally) nondeterministic behavior. sizeof is somewhat related in terms of establishing the memory used up by a value or pointer (4 bytes for a 32-bit address space), occurs at compile time so no runtime overhead.
- break, else, switch, case, return, for, continue, default, goto (BANNED), do, if, while are related to control flow.
- struct is for creating structs. enum is for creating enums. union is for creating unions.
- register is used for variables meant to be used so often they should be mapped directly to a register on the hardware for a performance boost (note the compiler is usually better at this and can ignore instances of this keyword).
- void enables generic programming for functions without a specified type until use.
- static is a scoping tool and a little complex...within a function it persists a value between function calls but it's still private to the function. Globally it provides encapsulation limiting the variable to the current file. Similar for functions where only other functions in the file can call it. Finally some weird array size trick.

8.2 The `volatile` Keyword

- **Definition:** A type qualifier informing the compiler that a variable's value may change at any time through external, undetectable means.

- **Prevents Optimization:** It disables aggressive compiler optimizations. Without it, a compiler might incorrectly assume a variable's state is safely cached in a CPU register, totally missing external hardware updates.
- **Forces Memory Access:** It guarantees the compiler will read the value from its exact memory location on every access, and immediately write any modifications back to memory.
- **Primary Use Case (Embedded Systems):** Absolutely essential for interacting with memory-mapped peripheral registers (e.g., I/O ports or hardware sensors) where values change due to external hardware events outside the program's control.

8.3 Some general tips yet again!

- Specify hexadecimal numbers with 0x..., 0b... for binary.
- Be careful with type casts involving void pointers as well as unary operator logic...
- Bitwise operations have arithmetic interpretations e.g. AND is the same as multiplying bits, OR is the same as adding (somewhat...), these interpretations can be a little tricky though. xor is its own inverse which can be useful (an interviewer mentioned using this to spawn and then remove a character's sprite each frame in a video game). Make sure to differentiate bitwise and logical operators carefully! (! vs , & vs &&, | vs ||, no logical xor but use an idiom: `!a != !b` vs caret for bitwise)
- Ternary, enums, and header awareness to be a cracked dev.
- Conditional compiling with #ifndef super specific but again maybe good for performance